



CS 231N Spring 2021 Practice Midterm Exam

Convolutional Neural Networks for Visual Recognition (Stanford University)



Scan to open on Studocu

CS 231N
Convolutional Neural Networks for Visual Recognition
Spring 2021 Practice Midterm Exam

April 29, 2021

Full Name: _____

SUNet ID (Not Number): _____

Question	Score
True/False (20 pts)	
Multiple Choice (40 pts)	
Short Answer (40 pts)	
Total (100 pts)	

Welcome to the CS231N Midterm Exam!

- The exam is 1 hour 30 minutes and is **double-sided**.
- No notes or electronic devices are allowed.

I understand and agree to uphold the Stanford Honor Code during this exam.

Signature: _____

Date: _____

Good luck!

This page is left blank for scratch work only. DO NOT write your answers here.

1 True / False (20 points)

Fill in the circle next to True or False, or fill in neither. Fill it in completely like this: ●. No explanations are required.

Scoring: Correct answer is worth 2 points. To discourage guessing, incorrect answers are worth -1 points. Leaving a question blank will give 0 points.

- 1.1 In order to normalize our data (i.e. subtract mean and divide by standard deviation), we typically compute the mean and standard deviation across the entire dataset before splitting the data into train/val/test splits.

- ☐ True
☐ False

SOLUTION:

False. We should only use training data for normalization, and never touch the validation and test data.

- 1.2 Suppose we have trained a softmax classifier (with weights W) that achieves 100% accuracy on our dataset. If we change the weights to $2W$, the classifier will maintain the same accuracy while have a smaller loss (cross entropy loss without regularization).

- ☐ True
☐ False

SOLUTION:

True. Multiplying the weight by 2 will make the normalized probabilities further apart, resulting in the probability of the correct class being higher than before (since we have 100% accuracy, the correct classes have the highest scores).

- 1.3 You currently have a small dataset containing images of handwritten digits 0 to 9, and would like to train a model for digit recognition. Each digit has the same number of images in the dataset. It is a good idea to augment the data and increase the size of the dataset by randomly flipping each image horizontally and vertically and adding the resulting image to the dataset.

- ☐ True
☐ False

SOLUTION:

False. The flipped representation of a digit isn't very meaningful and may confuse the model. Also, consider the digits 9 and 6. Flipping either of these over the horizontal axis will result in a image containing what looks like the other digit. This may affect the balanced dataset.

- 1.4 Recall that in Batch Normalization, the model has the capacity to learn the Beta and Gamma parameters such that at test time, these parameters can exactly undo the effects of mean centering and scaling by the standard deviation. However, in Layer Normalization, there does not exist Beta and Gamma parameters that can do this.

- ☐ True
☐ False

SOLUTION:

True: In Batch Normalization, there is a single mean and std-deviation that we learn across the dataset. Beta and Gamma can learn the inverses of this mean and std-deviation and thus exactly undo the centering and scaling. However, in Layer Normalization, each example in the dataset has its own mean and std-deviation that it is being transformed by, so it is impossible for a single Beta and Gamma to undo the transformation across all examples.

- 1.5 For an SVM classifier, initializing W and b to 0 will always result in all the elements in the final learned W to be the same.
- ☐ True
- ☐ False

SOLUTION:

False. If you write out the gradients, the gradient of each element at W is dependent on the different elements of the input vector x_i , so they are unlikely to be same. In fact, SVM is a convex problem and gradient descent is guaranteed to bring it to global optimum regardless of initialization.

- 1.6 For a binary classification task, using accuracy as a loss function directly over binary cross-entropy could yield better results in contexts where accuracy is your only metric of interest.
- Note: We define accuracy as $\frac{\text{count}(\text{round}(\hat{y})=y)}{N}$, given a real-valued prediction vector $\hat{y} \in [0, 1]^N$ and binary ground-truth vector $y \in \{0, 1\}^N$.
- ☐ True
- ☐ False

SOLUTION:

False. Accuracy is not a suitable loss function, as it is step-wise i.e. has a derivative of zero everywhere.

- 1.7 Let $\vec{z}$ be an n -dimensional vector and $s(\cdot)$ be the softmax function. Since softmax normalizes predictions to yield a probability distribution, its output does not change due to a scaling of the input, i.e. for all real numbers c , $s(z) = s(cz)$.

SOLUTION:

False. It's easy to verify this via counterexample ($s([1, 2, 3]) \neq s([2, 4, 6])$) or by expanding the expression for softmax, but it's essentially due to the fact that each element is taken to an exponential.

- 1.8 Recall that if network activations become saturated or collapse to zero, learning becomes challenging. Such problem can be alleviated with a learned normalization scheme (batch normalization, layer normalization, etc.) without changing the weight matrix of convolution or fully-connected layer.
- ☐ True
- ☐ False

SOLUTION:

True. If normalization is learned and applied correctly, activations can become better distributed.

- 1.9 Only recurrent neural networks have vanishing gradients and exploding gradients problems.
- ☐ True
 - ☐ False

SOLUTION:

False. Very deep neural networks can all potentially have these problems

- 1.10 PLACEHOLDER
- ☐ True
 - ☐ False

2 Multiple Choices (40 points)

Fill in the circle next to the letter(s) of your choice (like this: ●). No explanations are required. Choose ALL options that apply.

Each question is worth 4 points and the answer may contain one or more options. Selecting all of the correct options and none of the incorrect options will get full credits. For questions with multiple correct options, each incorrect or missing selection gets a 2-point deduction (up to 4 points).

2.1 Suppose you are using k - Nearest Neighbor method with L1 distance to classify images, which of the following will **NOT** affect the prediction accuracy?

- ☐ A: Increase the size of training set.
- ☐ B: Increase the value of k .
- ☐ C: Use L2 distance instead.
- ☐ D: Horizontally flip each training image and test image.
- ☐ E: All of the above will affect the prediction accuracy.

SOLUTION:

D. The distance metric won't be affected by horizontally flipping. Thus the prediction results won't change.

2.2 Recall that the hinge loss on i -th input data is defined as $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$, where y_i is correct label of i -th input, s_j is the score for j -th class and Δ is the margin of some non-negative value. Which of the following is/are true about the hinge loss?

- ☐ A: As long as $L_i = 0$ holds for each input, increase the margin won't change the value of L_i .
- ☐ B: The bias term b in the score calculation won't affect the value of L_i ,
- ☐ C: The more classes we would like to classify, the higher the hinge loss will be.
- ☐ D: All of the above.

SOLUTION:

B. Analysis: A. counterexample: $s_j = 2, s_{y_j} = 5$, change Δ from 2 to 4 will increase the loss; B. bias term is cancelled out; C. not necessarily, it depends on input and margin design.

2.3 Given n is the number of samples in the dataset, which of the following is the correct runtime complexity for test-time prediction for KNN and Linear SVM respectively?

- ☐ A: $O(1)$, $O(1)$
- ☐ B: $O(\log n)$, $O(1)$
- ☐ C: $O(n)$, $O(n)$
- ☐ D: $O(1)$, $O(n)$
- ☐ E: $O(n)$, $O(1)$

SOLUTION:

E.

2.4 Suppose the input to a **normalization layer** has dimensions (N, H, W, C) , where N is the number of data points (i.e. we have layer input $x_1, \dots, x_N$ and corresponding layer output $y_1, \dots, y_N$), and H, W, C are the height, width, and number of channels, respectively. For a specific input-output pair y_i, x_j where $i \neq j$, which of the following is/are true about the gradient matrix $\frac{\partial y_i}{\partial x_j}$?

- ☐ A: For a **batch** normalization layer, there are up to C nonzero terms.
- ☐ B: For a **batch** normalization layer, there are up to H^2W^2C nonzero terms.
- ☐ C: For a **batch** normalization layer, there are up to $H^2W^2C^2$ nonzero terms.
- ☐ D: For a **layer** normalization layer, there are up to C nonzero terms.
- ☐ E: For a **layer** normalization layer, there are up to H^2W^2C nonzero terms.
- ☐ F: For a **layer** normalization layer, there are up to $H^2W^2C^2$ nonzero terms.

SOLUTION:

B. Matrix size is $H^2W^2C^2$ but batch norm separates calculations for the channel dimension. Datapoints for layernorm are independent, $\frac{\partial y_i}{\partial x_j}$ is the zero-matrix.

2.5 You are training a CNN model that, after a number of convolutional and pooling layers, generates a volume of size $(H, W, C) = (8, 8, 1024)$. This is flattened and then fed through a 4096-neuron hidden layer and a final 200-neuron output layer (you are doing 200-class classification). You wish to represent the fully connected layers as convolutional layers that perform the same exact computations using square $F \times F$ filters. What would be the filter width (as given by parameter F) in the convolutional versions of the hidden and output FC layers respectively?

- ☐ A: 64, 200
- ☐ B: 64, 1
- ☐ C: 4096, 200
- ☐ D: 8, 1
- ☐ E: 8, 200

SOLUTION:

D.

2.6 PLACEHOLDER

- ☐ A: PLACEHOLDER
- ☐ B: PLACEHOLDER
- ☐ C: PLACEHOLDER
- ☐ D: PLACEHOLDER
- ☐ E: PLACEHOLDER

2.7 PLACEHOLDER

- ☐ A: PLACEHOLDER

- ☐ B: PLACEHOLDER
- ☐ C: PLACEHOLDER
- ☐ D: PLACEHOLDER
- ☐ E: PLACEHOLDER

2.8 PLACEHOLDER

- ☐ A: PLACEHOLDER
- ☐ B: PLACEHOLDER
- ☐ C: PLACEHOLDER
- ☐ D: PLACEHOLDER
- ☐ E: PLACEHOLDER

2.9 PLACEHOLDER

- ☐ A: PLACEHOLDER
- ☐ B: PLACEHOLDER
- ☐ C: PLACEHOLDER
- ☐ D: PLACEHOLDER
- ☐ E: PLACEHOLDER

2.10 PLACEHOLDER

- ☐ A: PLACEHOLDER
- ☐ B: PLACEHOLDER
- ☐ C: PLACEHOLDER
- ☐ D: PLACEHOLDER
- ☐ E: PLACEHOLDER

3 Short Answers (40 points)

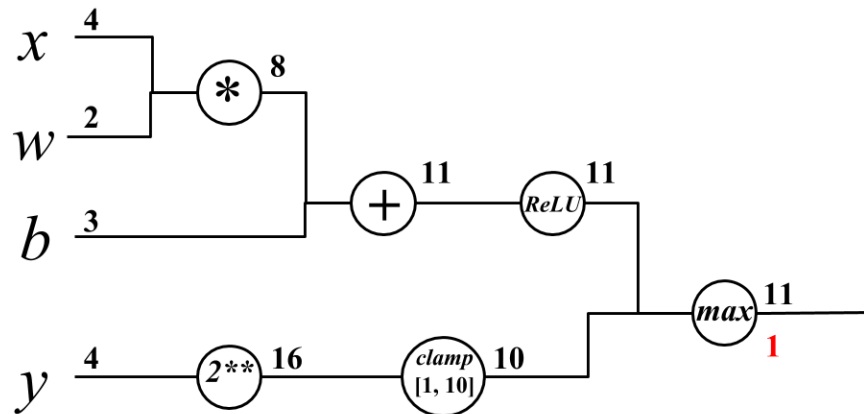
Please make sure to write your answer only in the provided space.

3.1 Computational Graphs (10 points)

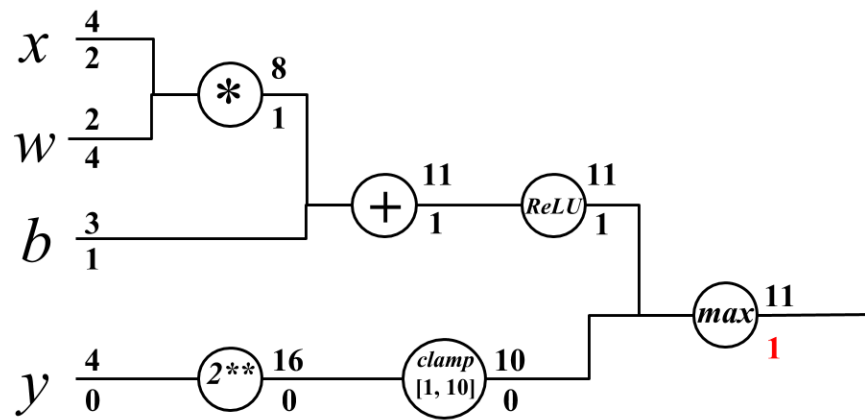
1. (5 points) In the following computational graph, the activations for the forward pass have been filled out (numbers above the connections). Please fill out the gradient for each activation; the last gradient is provided for you (i.e. 1).

- 2^{**} : exponential with base 2 (e.g. $2^{**3} = 8$)
- $\text{clamp}_{[a,b]}$: clamping input values to be no smaller than a and no larger than b (e.g. $\text{clamp}_{[1,10]}(12) = 10$); i.e.

$$\text{clamp}_{[a,b]}(x) = \begin{cases} a & \text{if } x < a \\ x & \text{if } a \leq x \leq b \\ b & \text{if } x > b \end{cases}$$



SOLUTION:



2. (5 points) A feature map X is passed to a convolutional layer with 3×3 kernel K (i.e. operation $*$) with no padding followed by a 2×2 max pooling layer (i.e. operation $\max$), resulting in a scalar output. Suppose the scalar output has a gradient of 1, what is the gradient at the position in X with value 1 (i.e. the underlined entry)?

$$\max \left(\begin{array}{|c|c|c|c|} \hline 4 & 6 & 3 & 4 \\ \hline \underline{1} & 7 & 3 & 4 \\ \hline 5 & 8 & 4 & 4 \\ \hline 2 & 7 & 4 & 4 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline 0 & -1/4 & 0 \\ \hline -1/4 & 1 & -1/4 \\ \hline 0 & -1/4 & 0 \\ \hline \end{array} \right)$$

$X \qquad K$

SOLUTION:

$$-\frac{1}{4}.$$

3.2 “Convolutions on Spatial Graphs” (15 points)

For many computer vision tasks such as image classification, it should not matter if an object is perfectly centered in an image or moved to the lower right corner. Convolutional neural networks achieve this so called translation-invariance with the help of 2D convolutions. Convolutions slide a filter W over a 2D input X to compute features independently of their location in the image. In order to extract features from a particular image patch $\tilde{X}$ the dot product $\tilde{X} \cdot W$ between $\tilde{X}$ and the overlapping filter W is computed. Sometimes, however it might be needed to mask out selected pixels when applying a convolutional filter. One such case are convolutions over 2D spatial graphs which need to respect the node connectivity as depicted on the left side of Figure 1.

Spatial graph definition: In this exercise, we define 2D spatial graphs $G = \langle X, E \rangle$ as follows. Each pixel on the image grid X defines a node x_{ij} in the graph. Edges E between pixel nodes define whether two nodes are connected and should influence each other or are unconnected and should not influence each other.

Spatial graph feature extraction: In order to extract features at node location x_{ij} from directly connected nodes, we center convolutional filter W over x_{ij} and compute the dot product, similarly to a 2D convolution. To prevent that unconnected nodes contribute to the feature extraction, we mask out nodes that are not connected to the center node x_{ij} with a binary mask M_{ij} (Figure 1, right). Specifically, we multiply filter W with mask M_{ij} element-wise before computing the dot product with $\tilde{X}$ to extract features or in other words activations Y :

$$Y = f(X, W, M) = \tilde{X} \cdot (W \odot M_{ij})$$

Each binary mask M_{ij} reflects the connectivity with respect to the center node and equals 1 if the overlapping pixel node is connected to the center node and 0 if it is not connected. Thus, while sliding the filter W across spatial graph locations ij , the binary masks M_{ij} change.

Now suppose we have the following 5×5 single-channel image X over which we define spatial graph $G = \langle X, E \rangle$, a 3×3 filter W and 3×3 mask M_{ij} :

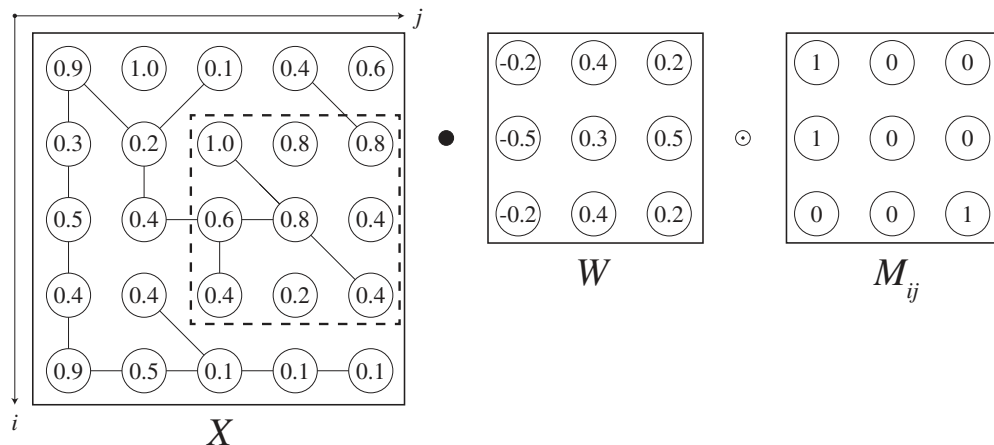


Figure 1: 5×5 single channel image X and 3×3 filter W and 3×3 mask M_{ij} .

1. (2 point) Calculate the activation at the dotted filter location by convolving filter W with image X (Ignore mask M_{ij} and the graph edges for now):

SOLUTION:

0.5

2. (2 point) Repeat the same calculation but this time only convolving over connected nodes in the graph with the help of M_{ij} :

SOLUTION:

-0.42

3. (2 points) What would be the binary mask M_{ij} if we would move the dotted box one pixel to the left?

SOLUTION:

The following binary matrix M_{ij} describes the connectivity at location "0.6":

0	0	0
1	0	1
0	1	0

4. (1 point) What would a "1" at the center of M_{ij} indicate?

SOLUTION:

A self-connection: The center node would be connected to itself and its features considered to compute the activation.

5. (2 points) Is the defined spatial graph feature extraction still translation-invariant in image-space X like a 2D convolution? Explain why or why not.

SOLUTION:

No, as M_{ij} depends on the filter location.

6. (2 points) The defined spatial graph feature extraction is translation-invariant within the space of spatial graphs, but still dependent on the relative position of connected nodes to the center node x_{ij} . Concretely, for example it matters whether a connected node is located to the right, left, bottom or top of a center node. How would you have to set the weights in filter W to remove this relative position dependency?

SOLUTION:

In its current form the weighting of a node is dependent on the relative location to the center node.

Thus, if we e.g. swap the top-left with the bottom-right node the result of the convolution will change. What we want to achieve for node-position-invariance is that nodes are processed independently of their relative location to the center node. One way to achieve this is to process every node feature with the same weights. Thus any filter W with the same numbers everywhere is a correct answer, e.g.:

0.2	0.2	0.2
0.2	0.2	0.2
0.2	0.2	0.2

7. (4 points) The defined spatial graph feature extraction $Y = f(X, W, M)$ only takes directly connected nodes into account (e.g. in Figure 1, X : feature y_{11} is only influenced by nodes x_{21} and x_{22}). If we apply this operation again on the previously extracted features $Z = f(Y, W, M)$, we can compute over indirectly connected second-order nodes (e.g. in Figure 1, X : z_{11} is now also influenced by nodes x_{31} , x_{32} , and x_{13}). To compute over third-order connections, fourth-order connections and so on, we can keep stacking these operations until we have taken the entire graph, i.e. all nodes for each feature location into account.

How many spatial graph feature extractors would we need to stack to take the entire graph in Figure 1 into account? What does the number of layers depend on and how could this be problematic? How could we potentially resolve this problem?

SOLUTION:

The answer is 7. The number of layers depends on the longest path in a connected graph component, i.e. parts of the graph with at least one edge between nodes. In our particular example the features propagate from both ends of the graph at the same time, meeting in the middle when the necessary number of propagation steps is reached. This is problematic because the longest path can change as the graph changes. One potential solution is to use recurrent neural networks to dynamically stop the propagation depending on the length of the longest path or just make sure the number of layers is enough to take the longest possible path across the entire spatial graph into account. Another problem is that nodes without any connections to other nodes or self connections will be zeroed out. Hence, to propagate those node features to later layers, one solution is to self-connect those nodes.

3.3 SHORT ANSWER QUESTION (X points)

PLACEHOLDER

This page is left blank for scratch work only. DO NOT write your answers here.

This page is left blank for scratch work only. DO NOT write your answers here.